

AI4005: Mini Project

Literature Survey

ANN Based Reformulations of Optimization Problems

ANIRUDH SRINIVASAN AJIT SHANKAR

List of Relevant Papers

- Base Paper: [An Evolutionary Neuro-Fuzzy C-means Clustering Technique](#)
 - **Rationale behind NFCM:**
 - NFCM is motivated by the idea of capturing patterns among similar data points through function approximators, particularly Multi-Layer Perceptron (MLP) networks.
 - In soft clustering algorithms like NFCM, the belongingness of a feature vector to a cluster is quantified using fuzzy numbers, representing the distribution of belongingness of a data point to the entire feature space.
 - The aim is to improve upon the traditional Fuzzy C-Means (FCM) algorithm by utilizing neural networks for function approximation, thereby learning the map between data points and their membership function (MF) values.
 - **Neuro-fuzzy formulation:**
 - NFCM utilizes MLP networks for function approximation.
 - The architecture of the MLP, weights, and biases of the MLP, along with cluster centers, act as decision variables (DVs) in the optimization problem.
 - NFCM reformulates the objective function of FCM to incorporate the output of the MLP, representing MF values.
 - The formulation involves solving an Integer Nonlinear Programming (INLP) problem to determine the optimal configuration of the ANN, transfer function, and number of clusters, maximizing the efficiency of clustering.
 - **Metric for cluster validation:**
 - NFCM employs the Partition Coefficient and Exponential Separation (PCAES) Index as a cluster validation metric to evaluate the quality of clusters formed by the algorithm.

- PCAES considers both compactness within clusters and separateness among clusters, providing a comprehensive measure of clustering efficiency.
 - Optimal configuration of ANN and optimal cluster number:
 - NFCM formulates an optimization problem to determine the optimal values of nodes and layers of the ANN, and number of clusters.
 - The formulation includes constraints to ensure the presence of at least one hidden layer in the ANN and a minimum of two clusters.
 - An evolutionary optimizer, specifically a binary coded elitist Genetic Algorithm (GA), is employed to solve the optimization problem and find global optimal solutions.
 - The architecture of the ANN and transfer function obtained are stored in a database for efficiency and redundancy checking.
- **Difference between FCM and NFCM:**
 - NFCM decomposes the optimization problem into two parts: maximizing PCAES Index and minimizing the FCM objective function, using ANN mapping.
 - NFCM reduces the number of decision variables compared to FCM, leading to computational efficiency.
 - A population-based GA is utilized to solve both optimization problems iteratively.
- **Implementation of NFCM:**
 - NFCM algorithm involves an outer loop for optimizing the ANN architecture and an inner loop for optimizing MF values and cluster centers.
 - Evolutionary operations like selection, crossover, and mutation are performed to find global optimal solutions.
 - Interior-Point method is used for converging to local optima in the global basin after obtaining initial solutions from the inner loop of GA.
- Survey Papers
 - (Anirudh) [Global Deterministic Optimization with Artificial Neural Networks Embedded](#)
 - **Objective and Motivation:**
 - The paper aims to optimize problems embedded with ANNs deterministically on a global scale, utilizing surrogate-based modeling for complex systems where traditional modeling is challenging.
 - It highlights the limitations of local optimization methods and the need for global optimization to address nonconvexities in the activation functions of ANNs.

■ **Methodology:**

- The approach uses McCormick relaxations within a reduced-space optimization framework. This involves capturing both convex and concave properties of non-linear activation functions in ANNs.
- A specific solver, MAiNGO, developed in-house, is employed for solving these relaxed problems globally.

■ **Activation Function Handling:**

- Special attention is given to the mathematical treatment of non-linear activation functions like the hyperbolic tangent. The paper discusses creating convex and concave envelopes for these functions to facilitate global optimization.
- This is crucial because the nonconvex nature of standard activation functions can lead to suboptimal solutions if not handled properly.

■ **Implementation Details:**

- The optimization problems are described in both full-space (all variables and equations) and reduced-space (reduced number of variables and constraints) formulations.
- The paper explains the use of branch-and-bound algorithms enhanced by the McCormick relaxation techniques to handle the ANN embedded optimization tasks efficiently.

■ **Performance and Applications:**

- Several application examples are provided, including a fermentation process and a compressor plant optimization, to demonstrate the effectiveness of the proposed method.
- Performance comparisons with a general-purpose optimization solver, BARON, show favorable results in terms of computational efficiency.

■ **Technical Challenges and Solutions:**

- The authors discuss the challenges of dealing with large-scale problems and the importance of tight variable bounds to ensure effective relaxation propagation.
- Solutions include the automatic construction of McCormick relaxations and leveraging problem structure (like sparsity and decomposition) to reduce computational demands.

■ **Future Directions:**

- The paper suggests potential extensions to other types of complex systems, including mixed-integer problems and dynamics involving multiple interacting ANNs.
 - There is also a focus on further refining the relaxation techniques and exploring their applications in more diverse settings.
- (Anirudh) [Clustering high-dimensional data: A survey on subspace clustering, pattern-based clustering, and correlation clustering](#)

- **Overview of High-Dimensional Data Challenges:**

- High-dimensional data presents unique challenges for clustering because traditional similarity measures often become less effective as dimensionality increases. The paper reviews various methods to address these issues, focusing on subspace clustering, pattern-based clustering, and correlation clustering.

- **Subspace Clustering:**

- Subspace clustering aims to find clusters in subspaces of the full feature space, which may reveal hidden structures that are not apparent in the full-dimensional space. Techniques involve identifying dense regions in various subspaces that may not be dense in the original high-dimensional space.
 - Methods discussed include projected clustering and axis-parallel subspace clustering, using algorithms like CLIQUE and SUBCLU, which reduce the dimensionality problem by focusing on lower-dimensional projections.

- **Pattern-Based Clustering:**

- This approach looks for clusters where data exhibits certain patterns or behaviors across a subset of dimensions, not necessarily limited to dense areas. Biclustering is a specific form of pattern-based clustering that simultaneously clusters both rows and columns of a data matrix.
 - Algorithms like Bimax and CC (Cheng and Church) look for homogeneity in matrix blocks, capturing patterns where the data is more similar or behaves more coherently than in the full dataset.

- **Correlation Clustering:**

- Correlation clustering tries to find clusters where the feature values are correlated, which may not imply close distances or high density. This approach can identify complex relationships among features that traditional clustering methods might miss.
 - The focus is on discovering groups where data points exhibit correlated behavior across dimensions, which can be particularly

useful in scenarios like gene expression data analysis.

■ **Heuristics and Assumptions:**

- The authors discuss the importance of heuristics and assumptions in the development of clustering algorithms for high-dimensional data. They explore the implications of different assumptions on the outcomes of clustering algorithms and stress the necessity of understanding these effects for accurate data analysis.

■ **Comparative Analysis and Applications:**

- The paper compares several algorithms across the different clustering approaches, evaluating their effectiveness and applicability in various scenarios.
- It provides insights into which methods are better suited for specific types of data challenges, such as sparse data, data with hidden patterns, or data requiring dimensionality reduction.

○ (Ajit) [Artificial Neural Networks Based Optimization Techniques: A Review](#)

■ **Introduction and Background:**

- The paper emphasizes the critical role of optimization in enhancing the performance of ANNs.
- It highlights how different optimization techniques can be adapted to improve the training parameters of ANNs, aiming for more efficient neural architectures.

■ **Optimization Techniques Overview:**

- Various algorithms such as Genetic Algorithms (GA), Particle Swarm Optimization (PSO), and newer methods like Whale Optimization Algorithm (WOA) and Lightning Search Algorithm (LSA) are discussed.
- These techniques help in optimizing the structural parameters of ANNs, such as weights and biases, by simulating natural processes or physical phenomena.

■ **Particle Swarm Optimization (PSO):**

- PSO is detailed with its application in optimizing ANN parameters by adjusting the velocity and position of particles (potential solutions) within the search space. This mimics the social behavior of birds or fish.
- Mathematically, PSO updates particles' velocities and positions using equations that consider the best-known positions of the particles and the global best-known position.

■ **Genetic Algorithm (GA):**

- GA simulates the process of natural selection where the fittest individuals are chosen for reproduction to produce offspring of the next generation.
- It uses operators like selection, crossover, and mutation to evolve solutions toward the best fitness values.
- The selection process focuses on choosing the best-performing individuals, crossover combines the features of parent solutions, and mutation introduces random changes to offspring.

■ **Challenges and Solutions:**

- The paper discusses the challenges of convergence in high-dimensional spaces and the risk of getting trapped in local minima.
- Advanced techniques like hybrid optimization methods combining features of two or more standard algorithms (e.g., hybrid PSO-GA) are suggested to overcome these challenges.

■ **Application in Energy Systems:**

- Specific applications of optimized ANNs in energy management, such as in virtual power plants, are explained.
- Here, optimization algorithms help to fine-tune ANNs that forecast and manage energy distribution and consumption efficiently.

■ **Mathematical Formulations:**

- The paper includes mathematical formulations for each optimization algorithm, explaining how each adapts to the specific requirements of neural network training.
- For example, in PSO, the velocity update equation is vital for guiding particles toward optimal solutions.

■ **Results and Comparisons:**

- Comparative analysis shows how different optimization techniques affect the performance and training speed of ANNs.
- The results are crucial for determining the most effective algorithm for a specific application, considering factors like convergence rate and accuracy.

■ **Future Directions:**

- The review suggests exploring more adaptive and self-tuning optimization algorithms that can dynamically adjust their parameters in response to the changing landscape of the problem space.

- (Ajit) [Comparison of hierarchical clustering and neural network clustering: an analysis on precision dominance](#)

- **Objective of Study:**

- The study aims to determine the precision dominance between NNC and HC when clustering a large dataset of water quality variables, which can help in reducing data complexity without losing essential information.

- **Neural Network Clustering (NNC):**

- Methodology: Utilizes a Self-Organizing Map (SOM) to learn the distribution and topology of variables in an input space.
- SOM Training: A batch algorithm is employed where the entire dataset is presented before any weights are updated. The algorithm identifies a "winning" neuron for each input vector and adjusts weight vectors towards the average position of all input vectors for which they are winners.

- **Mathematical Expression for Updating Weights:**

- $\text{new weight} = \sum (\text{current weight} + \text{learning rate} * (\text{input vector} - \text{current weight}))$
- The training helps the SOM to group variables based on their similarity, which is visualized through various plots like sample hits, neighbor weight distances, and weight planes.

- **Hierarchical Clustering (HC):**

- Approach: Builds a hierarchy of clusters where each node represents a cluster of similar characteristics data points.
- Dendrogram Construction: Utilizes a dissimilarity matrix to measure distances between pairs of variables and a linkage matrix to define the hierarchical relationships.
- Cophenetic Coefficient Calculation: Used to validate the clustering by measuring the correlation between the actual pairwise distances and those implied by the hierarchical clustering.
 - $\text{cophenetic coefficient} = \sum (\text{actual distance} * \text{hierarchical distance}) / \sqrt{(\sum (\text{actual distance}^2) * \sum (\text{hierarchical distance}^2))}$

- **Comparative Analysis:**

- Both clustering methods are analyzed for their ability to categorize system predictors and inputs with a precision probability of 0.8.
- The integration of HC's linkage and dissimilarity algorithms with NNC is suggested to provide more reliable clustering outcomes than using either method independently.

- The study observes that altering the map size in SOM affects the association of input weights to neurons, hence changing the cluster formation.

■ **Results and Discussion:**

- The study finds that combined application of HC and NNC is more effective than individual methods.
- It suggests that while NNC provides a good visual understanding of the data, HC offers a more rigorous computational approach to determine the exact number of clusters and their boundaries.
- The clusters identified through HC are validated against the cophenetic coefficient, confirming that the hierarchical links accurately represent the distances between data points.

• Other Relevant Papers

- (Anirudh) [Optimization Model of K-Means Clustering Using Artificial Neural Networks to Handle Class Imbalance Problem](#)

■ **Problem Statement:**

- Class imbalance in data can skew the performance of clustering algorithms towards the majority class, leading to poor model accuracy, especially for the minority class.
- This is a common issue in fields like anomaly detection and clustering where the minority class might be of significant interest.

■ **Proposed Solution:**

- The authors propose using a perceptron feed-forward neural network to optimize the initial centroid determination in K-Means clustering.
- This approach aims to adjust centroids better to handle class imbalances effectively.

■ **Methodology:**

- Neural Network for Centroid Initialization: Instead of randomly initializing centroids, the paper uses a neural network to predict the initial positions of centroids based on the input data.
- This is thought to provide a more informed starting point for K-Means, potentially leading to better clustering performance, especially in imbalanced datasets.

■ **Mathematical Model:**

- The K-Means algorithm typically minimizes the squared-error criterion, which is the sum of squared distances between data points and their respective cluster centroids.

- **Optimization via Neural Network:**

- Input: Number of clusters (k) and a dataset with n objects.
- Output: Set of k clusters minimizing the squared-error criterion.
- Steps:
 - Step 1: Use a perceptron neural network to calculate the initial cluster centers based on the provided data.
 - Step 2-5: Standard K-Means clustering iterations to assign objects to the nearest cluster and update centroids based on current cluster memberships.

- **Experimental Setup:**

- Datasets from the UCI Machine Learning Repository were used to evaluate the effectiveness of the proposed method.
- The training involves setting the perceptron with initial weights and biases, defining a learning rate, and iteratively adjusting the network to minimize error in centroid placement.

- **Results:**

- The proposed method showed an improvement in handling class imbalances by optimizing the centroid initialization process.
- This led to a more balanced distribution of data points among clusters compared to traditional K-Means.

- **Conclusion:**

- By integrating a neural network into the centroid initialization process of K-Means clustering, the authors were able to address the class imbalance problem effectively, leading to more equitable clustering results.

- (Anirudh) [Joint Learning of Unsupervised Dimensionality Reduction and Gaussian Mixture Model](#)

- **Introduction and Objective:**

- The study addresses the challenge of dimensionality reduction in high-dimensional data and its impact on the performance of learning models such as GMM.
- Traditional approaches often perform DR and GMM training independently, which may not yield the optimal subspace for subsequent modeling tasks.

- **Methodological Approach:**

- The paper proposes a joint learning framework where DR and GMM are optimized simultaneously.
- This approach uses the Mixture of Factor Analyzers (MFA) model, assuming a common factor loading across all components, which acts as the dimensionality reduction matrix.

■ Mathematical Model:

• Gaussian Mixture Model:

- $P(y; \theta) = \sum_{i=1}^g (\pi_i * \prod_{j=1}^n P(y_j | x_{ij}))$
- Where
 - y is the data point,
 - θ includes mixture weights π_i , means μ_i , and covariance matrices Σ_i , and
 - x is a latent variable.

• Factor Analysis Representation:

- $\Sigma_i = W_i * W_i^T + \Psi_i$
- Where
 - Ψ_i is a diagonal matrix and
 - W_i contains the factor loadings.

• Common Loading Matrix:

- $\mu_i = Z * \zeta_i$
- $\Sigma_i = Z * \eta_i * Z^T + \Psi_i$
- Here Z is a common loading matrix across all components, reducing dimensionality by mapping high-dimensional data to a lower-dimensional latent space.

■ Optimization via EM Algorithm:

- The Expectation-Maximization (EM) algorithm is adapted to jointly optimize the dimensionality reduction matrix and the parameters of the GMM.
- E-step: Computes the posterior probabilities based on current parameter estimates.
 - $\rho_i(y_j; \theta) = (\pi_i * \phi(y_j; Z * \zeta_i, Z * \eta_i * Z^T + \Psi_i)) / \sum_{h=1}^g (\pi_h * \phi(y_j; Z * \zeta_h, Z * \eta_h * Z^T + \Psi_i))$
- M-step: Updates the parameters to maximize the expected log-likelihood.
 - $Z(\text{new}) = (\sum \text{over all } i \text{ of } (Z_{1_i})) * (\sum \text{over all } i \text{ of } (Z_{2_i}))^{(-1)}$
 - Where Z_{1_i} and Z_{2_i} are intermediate variables dependent on the responsibilities and data points.

- **Experimental Validation:**
 - The paper includes experiments on synthetic and real-world datasets to demonstrate the effectiveness of the proposed method.
 - Comparisons are made with traditional DR methods followed by GMM and other joint learning methods, showing superior clustering performance in terms of error rates, Bayesian Information Criterion (BIC), and Adjusted Rand Index (ARI).
- **Conclusions:**
 - The integration of dimensionality reduction with GMM parameter estimation in a single joint learning framework provides significant improvements over methods where these two steps are performed independently.
 - This approach is particularly beneficial in unsupervised learning scenarios like clustering where no class labels are available to guide the dimensionality reduction.
- (Anirudh) [Gaussian mixture model with feature selection: An embedded approach](#)
- Literature Review
 - **Fuzzy Centroid Determination:**
 - Utilizes fuzzy approach for determining centroids in k-means algorithm.
 - Despite offering an alternative, fuzzy approach is noted to lack accuracy for large datasets.
 - **Cost Function and Distance Measure Enhancement:**
 - Proposes a new cost function and distance measure based on co-occurrence of values to address limitations of k-means with numeric data.
 - Intends to improve the description of cluster centers for better clustering results.
 - **Fuzzy C-Means (FCM) Integration:**
 - Integrates clustering results obtained from Fuzzy C-Means (FCM) into a judgment matrix.
 - Utilizes iterative partitioning of the judgment matrix to identify optimal cluster number and refine final results.
 - **Neural Networks Approach for Imbalance Problems:**
 - Introduces a modified backpropagation method to accelerate learning rate for two-class imbalance problems.

- Focuses on equalizing error reduction for both majority and minority classes.
- **Correction of Clustering Class Errors:**
 - Addresses errors in clustering class placement, particularly focusing on class imbalance.
 - Proposes a modified learning algorithm to rectify the issue and improve clustering accuracy.
- **Modified Back Propagation:**
 - Presents a modified backpropagation algorithm to prevent the neglect of minority class during training.
 - Aims to enhance convergence speed of neural networks while maintaining balance between classes.
- **Deep Neural Network Training for Imbalance Handling:**
 - Implements deep neural network training to handle imbalanced datasets.
 - Focuses on capturing classification errors from both majority and minority classes equally.
- **Under Sampled K-Means for Imbalance:**
 - Proposes an under-sampled k-means technique to address class imbalance.
 - Intelligently removes noisy and weak instances from overwhelming (majority) class for improved clustering.
- **Joint Learning of Unsupervised Dimensionality Reduction and Gaussian Mixture Model:**
 - Reduces the dimension of the input data points to extract only the relevant features for better clustering.
 - Utilizes Mixture of Common Factor Analyzers (MCFA) to generate the dimensionality reduced matrix.
 - Could be combined with ANN based classification technique to produce better predictions of the mean and covariance matrices in this reduced space.
- **Visual K-Means for Skewed Data:**
 - Notes that resulting clusters from k-means are relatively uniform in size despite input data variation.
 - Utilizes Visual K-Means algorithm for skewed data sources to effectively handle imbalanced datasets.

■ Main Algorithm

- **Step 1: Initialization**
 - Initialize weights and bias:
 - Initialize the weights and bias of the perceptron network. These parameters are crucial for the learning process.
 - Set learning rate:
 - Define the learning rate, denoted by α . It controls the magnitude of weight updates during the learning process.
 - Set input according to dataset:
 - Prepare the input data according to the dataset. Each data point represents an object in the dataset.
 - Set target according to cluster of data from dataset:
 - Determine the target output for each data point based on the cluster to which it belongs. This information is used to train the perceptron.

- **Step 2: Perceptron Training**
 - Loop until convergence:
 - Repeat the following steps until convergence criteria are met.

- **Step 3: Iterate Through Training Pairs**
 - For each training pair $s:t$:
 - Iterate through each training pair, where 's' represents the input data (object) and 't' represents the target output (cluster).

- **Step 4: Set Activations of Input Units**
 - Set activations of input units:
 - Set the activation of each input unit to the corresponding feature value of the input data.
 - For each input feature x_i , set its activation as $x_i = s_i$, where ' s_i ' is the value of the i th feature of the input data.

- **Step 5: Compute Response of Output Unit**
 - Compute response of output unit:
 - Compute the response of the output unit, which represents the predicted cluster for the input data.
 - Calculate the weighted sum of inputs along with the bias term:
 - $y_{in} = b + \sum(w_i * x_i)$, where 'b' is the bias and 'w_i' are the weights.
 - Apply an activation function to determine the output value 'y'.

- **Step 6: Update Weights and Bias**

- Update weights and bias if an error occurred for this pattern:
- If the predicted output 'y' does not match the target output 't', update the weights and bias to minimize the error.
- Use the perceptron learning rule to adjust weights and bias:
- If $y \neq t$, update weights and bias:
 - $w_i (\text{new}) = w_i (\text{old}) + \alpha * t * x_i$
 - $b (\text{new}) = b (\text{old}) + \alpha * t$
- Otherwise, keep the weights and bias unchanged.

- **Step 7: Test Stopping Condition**

- Test stopping condition:
 - Check if any weights have changed during the iteration. If no weights have changed, stop the training process as convergence is achieved. Otherwise, continue to the next iteration.

- (Ajit) [A robust adaptive clustering analysis method for automatic identification of clusters](#)

- **Objective and Challenges Addressed:**

- The paper aims to optimize cluster analysis by identifying the optimal number of clusters and ensuring robust clustering results without prior knowledge of the data structure.
- The approach is adaptive, working across various clustering algorithms like Fuzzy C-Means and k-means.

- **Methodology - Robust Adaptive Clustering Method:**

- The method integrates results from multiple initializations of a clustering algorithm to form a judgment matrix, which helps determine the most consistent clustering configuration.

- **Mathematical Formulations:**

- **Cluster membership and judgment matrix:**

- The observation matrix OC is defined such that each element o_{ij} indicates whether points i and j belong to the same cluster.
- This is determined by clustering data with varying numbers of clusters.
- The judgment matrix J sums all the observation matrices, reflecting the overall agreement across different clustering results on point associations.

- $J = \text{sum of all OC from } C = 2 \text{ to } k$

- **Iterative Graph Partitioning:**

- This process refines the clustering by interpreting the judgment matrix as a graph and partitioning this graph to identify clusters.
- The graph-partitioning algorithm reduces connections iteratively until the most coherent cluster divisions remain stable.

- **Algorithm Implementation Details:**

- Initial Clustering: Perform multiple runs of a chosen clustering algorithm (like FCM or k-means) with different initial cluster numbers.
- Result Integration: Aggregate the results using the observation and judgment matrices.
- Graph Partitioning: Apply a graph partitioning algorithm on the judgment matrix to finalize the cluster configuration.

- **Experimental Validation:**

- The method was tested on both synthetic and real-world datasets (including commonly used datasets like Iris and Wine) to demonstrate its effectiveness in producing stable and accurate clusters compared to standard methods.
- The results show that the proposed method can adaptively and robustly identify the correct number of clusters and achieve high-quality clustering.

- **Adaptive and Flexible Implementation:**

- The approach is adaptable to different clustering algorithms without requiring modifications, showing flexibility across various applications, including mesh segmentation and market segmentation.

- **Mathematical Justification:**

- The strength of the method lies in its ability to repeatedly evaluate and integrate clustering results through mathematical operations on matrices, ensuring that the final clustering solution is not only statistically valid but also robust against outliers and varying data distributions.

- (Ajit) [Missing value imputation through shorter interval selection driven by Fuzzy C-Means clustering](#)

■ **Problem Statement:**

- Missing data is a common problem that decreases the quality of datasets and affects the outcomes of data analysis.
- The paper focuses on using clustering to improve imputation accuracy by focusing on the most relevant feature values from selected instances.

■ **Methodology - Fuzzy C-Means Clustering:**

- The authors employ Fuzzy C-Means (FCM) clustering to handle the imputation.
- FCM allows each data point to belong to multiple clusters with varying degrees of membership, which is ideal for imputing missing values as it provides a soft classification of the data.

■ **Short Interval Selection:**

- The core idea is to select shorter intervals of feature values based on cluster membership values for imputation.
- This method assumes that an imputation from a shorter range of similar values is more precise than one from a broader range.

■ **Mathematical Formulation:**

● **Cluster Membership Calculation:**

- $\mu_{ik} = 1 / \sum (||R_i - v_j||^2 / (m-1))^{(-1)}$, $j=1$ to n
- Here,
 - μ_{ik} is the membership of the i -th data point in the k -th cluster,
 - R_i is the data point,
 - v_j is the centroid of the j -th cluster, and
 - m is a fuzziness parameter greater than 1, typically set to 2.
- This formula calculates the inverse distance of R_i from all cluster centroids, normalizing it to determine the membership grade.

● **Centroid Update:**

- $v_k = \sum (\mu_{ik}^m * R_i) / \sum (\mu_{ik}^m)$, $i=1$ to n
- The centroid v_k for each cluster is recalculated as the weighted average of all points, where weights are the membership grades raised to the power of m .

■ **Imputation Process:**

- For imputing missing values, the method selects the cluster with the highest membership value for the data point with missing attributes and uses the centroid of this cluster or the mean of the

closest points within the cluster to estimate the missing values.

■ **Experimental Validation:**

- The method was tested on multiple UCI datasets, showing improvement in imputation accuracy compared to other state-of-the-art methods.
- The evaluation was based on how well the imputed data supported accurate classifications, using metrics like classification accuracy before and after imputation.

■ **Advantages:**

- The approach is particularly advantageous in scenarios where data points naturally group into distinct clusters, allowing for more accurate estimations of missing values based on local similarities within data.